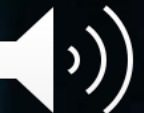




COM3021/COMM115 – Data Science at Scale

Consistency and consensus

DR HUGO BARBOSA



Introduction

- When it comes to distributed system, many things can go wrong
... and they will....
 - Letting a system fail entirely might not be acceptable in many (most?) cases
 - Building reliable, fault-tolerant systems from unreliable components is challenging
Algorithms, abstractions and protocols (e.g., transactions in databases)
-

The CAP theorem

Distributed systems? Choose two:

- Consistency

Every read receives the most recent write or an error.

- Availability

Every request receives a (non-error) response.

- Partition tolerance

The system continues to operate even if parts of the network become disconnected

The CAP theorem

- More precisely:

If there is a network partition, you have to choose between availability and consistency

A.C.I.D. properties

- Atomicity

Each statement in a transaction (to read, write, update or delete data) is treated as a single unit.

- Consistency

Ensures that transactions only make changes to tables in predefined, predictable ways

- Isolation

When multiple users are reading and writing from the same table all at once, isolation of their transactions ensures that the concurrent transactions don't interfere with or affect one another

- Durability

- Ensures that changes to your data made by successfully executed transactions will be saved, even in the event of system failure.
-

Eventual consistency

Eventual consistency

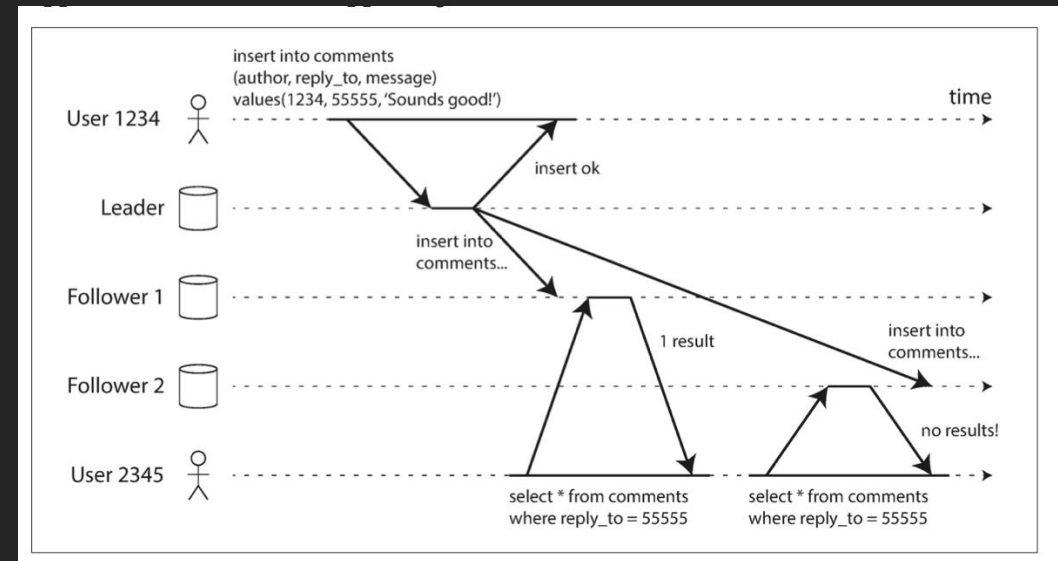
- At a given point in time, some data nodes in a distributed system will have an outdated version of the data

Write queries will arrive at different nodes at different times

- Most replicated data systems will provide an eventual consistency

All nodes will have the same data

Read requests will eventually return the same value



Eventual consistency

- Can be seen as a convergence

All the nodes will converge towards the same state

- How long will it take?

Until then, reads could return anything

- These transient inconsistencies are hard to detect

They tend to appear only in case of system faults or high volume of concurrent operations

Linearizability

Linearizability

- An eventually consistent database may give different answers to the same question

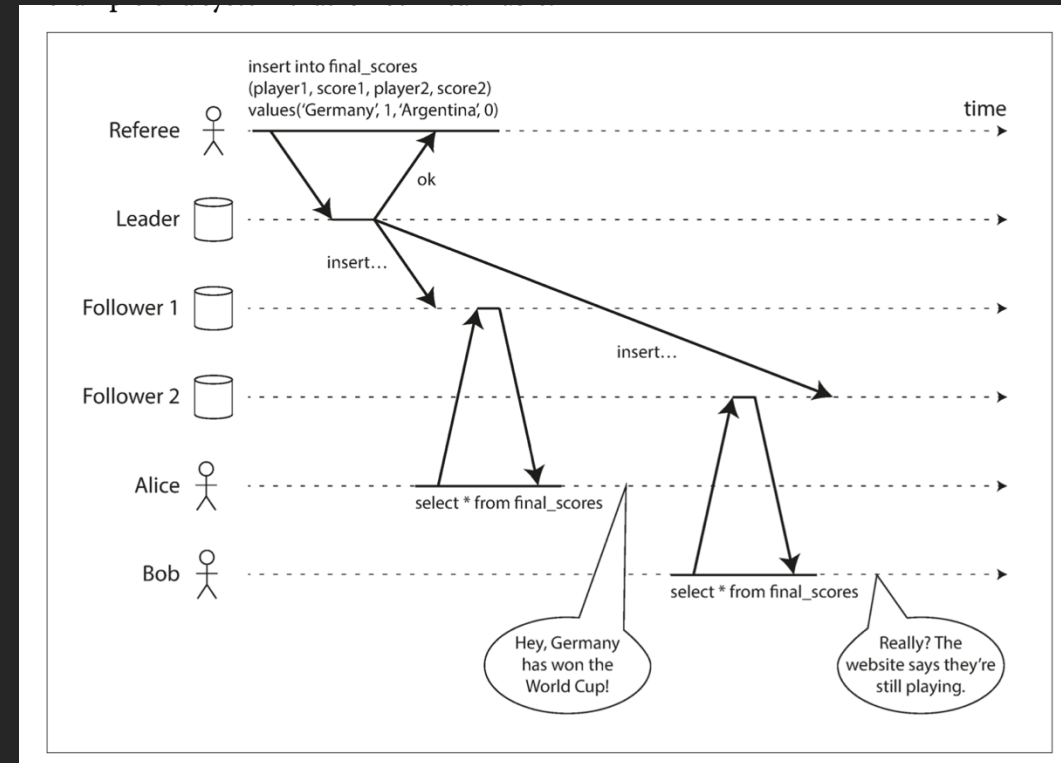
Impacts the reliability of the system

- Ideally, a distributed database should behave, from the user perspective, as if there is only one version of the data

The most current one

Linearizability

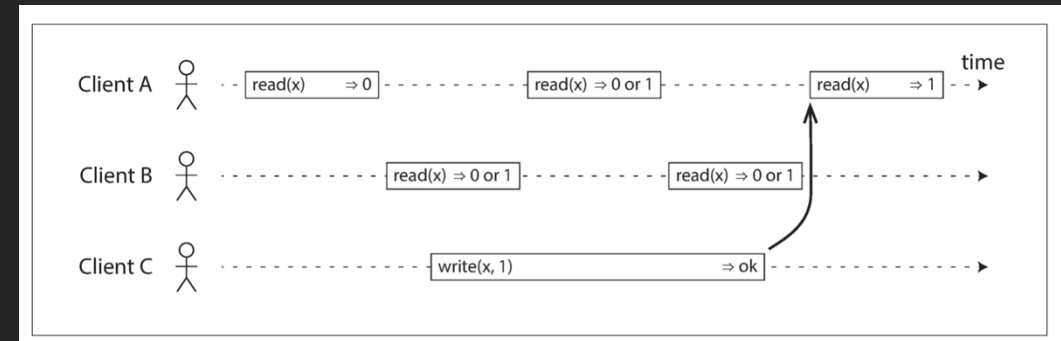
- The system should appear as if all operations on it are atomic and that there is one single copy of the data
- As soon as a client completes a write, all clients must be able to see that data



A non-linearizable system

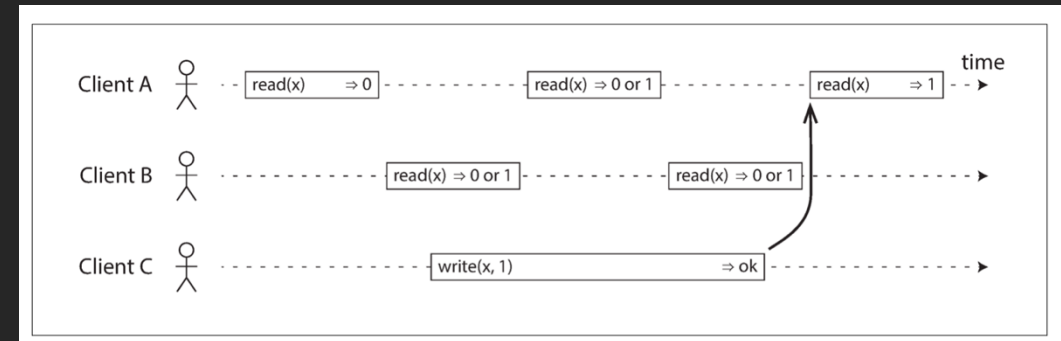
Linearizability

- *Let's look at this other example from the client perspective*
- *A register can be seen as a document or a row in a relational database*
- *There are two possible operations:*
 - *$\text{read}(x) \Rightarrow v$ means the client requested to read the value of register x , and the database returned the value v .*
 - *$\text{write}(x, v) \Rightarrow r$ means the client requested to set the register x to value v , and the database returned response r (which could be ok or error).*



Linearizability

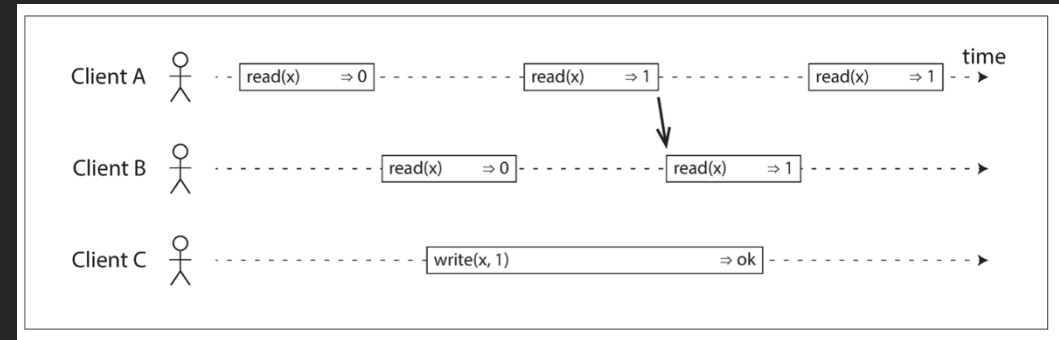
- Each bar represents the duration of the operations
- The value is initially 0
- Clients A and B perform multiple concurrent reads while client C will perform a write to set it to 1
 - The first read by client A is performed before the write starts: $\Rightarrow 0$
 - The last read by client A after the write is completed: $\Rightarrow 1$
 - Read operations that overlap with the write?
Result: $\Rightarrow 0$ or 1



Still not quite there yet

Linearizability

- We need an additional constraint
- In a linearizable system, if one client reads 1, all subsequent clients should also read 1



Linearizability

- What kind of systems linearizability is crucial?
 - *A system that uses single-leader replication needs to ensure that there is indeed only one leader*
 - *Constraints and uniqueness guarantees*
 - No two entries with the same Unique ID
 - No passengers booking the same seat on a flight
-

Linearizability and the CAP theorem

- If your application requires linearizability, and some replicas are disconnected:
These replicas cannot process requests while they are disconnected (sacrificing the availability)
 - Applications that don't require linearizability can be more tolerant of network problems
 - Linearizability has performance impacts
 - Linearizability is not guaranteed by many distributed database systems in favour of performance
-

Linearizability and network delays

- If you want linearizability, the response time of read and write requests is at least proportional to the uncertainty of delays in the network
 - In a network with highly variable delays the response time of linearizable reads and writes is inevitably going to be high
-

Consensus

- Getting several nodes to agree on something

Leader election

Who is the new leader in case of a failure in a single-leader architecture?

Atomic commit

Transaction may fail on some nodes but succeed on others (all nodes have to agree on the outcome of the transaction)

Consensus

- Atomicity can be easily implemented on single-node databases
 - What if multiple nodes are involved in a transaction (e.g., a partitioned database)?
 - It is not sufficient to simply send a commit to all of the nodes and independently commit the transaction on each one*
 - *Commits could succeed on some nodes and fail on other nodes*
 - Some nodes may detect a constraint violation or conflict
 - Some of the commit requests might be lost in the network
 - Some nodes may crash
-

Consensus

- Two-phase commit (2PC)

An algorithm for achieving atomic transaction commit across multiple nodes

The commit/abort process in 2PC is split into two phases using a new component called coordinator or transaction manager

- When the application is ready to commit it sends a prepare request to each of the nodes*
 - If all participants reply “yes,” the coordinator sends out a commit request
 - If any of the participants replies “no,” the coordinator sends an abort request to all nodes

